

**Московский государственный технический
университет им. Н.Э. Баумана**

Факультет Радиотехнический
Кафедра “Системы обработки информации и управления”

Курс «Парадигмы и конструкции языков программирования»

Отчет по рубежному контролю №2
Вариант №12

Выполнил:
студент группы РТ5-31Б:
Мусаев Максим
Шахинович
Подпись и дата:

Проверил:
преподаватель каф. ИУ5
Гапанюк Юрий
Евгеньевич
Подпись и дата:

Москва, 2025 г.

Постановка задачи

Провести рефакторинг текста программы рубежного контроля №1 таким образом, чтобы он был пригоден для модульного тестирования. Для изменённой программы рубежного контроля №1 разработать модульные тесты с применением методологии TDD и фреймворка unittest, включающие минимум три теста.

Текст программы

art_gallery.py

```
from prettytable import PrettyTable
from typing import List, Dict, Any, Optional

class PictureRepository:
    def __init__(self):
        self._pictures: List['Picture'] = []

    def add(self, picture: 'Picture') -> int:
        picture.id = len(self._pictures)
        self._pictures.append(picture)
        return picture.id

    def get_by_id(self, picture_id: int) -> Optional['Picture']:
        if 0 <= picture_id < len(self._pictures):
            return self._pictures[picture_id]
        return None

    def delete(self, picture_id: int) -> bool:
        if 0 <= picture_id < len(self._pictures):
            self._pictures.pop(picture_id)
            self._update_ids()
            return True
        return False

    def get_all(self) -> List['Picture']:
        return self._pictures.copy()

    def find_by_name(self, name: str) -> List['Picture']:
        return [p for p in self._pictures if p.name == name]

    def find_by_country(self, country: str) -> List['Picture']:
        return [p for p in self._pictures if p.country == country]

    def find_by_year(self, year: int) -> List['Picture']:
        return [p for p in self._pictures if p.year == year]

    def find_by_price_range(self, from_price: int, to_price: int) ->
List['Picture']:
```

```

        return [p for p in self._pictures if isinstance(p.price, int) and
from_price <= p.price <= to_price]

    def find_by_author(self, author_id: int) -> List['Picture']:
        return [p for p in self._pictures if author_id in p.authors_ids]

    def clear(self):
        self._pictures.clear()

    def _update_ids(self):
        for i, picture in enumerate(self._pictures):
            picture.id = i

class AuthorRepository:
    def __init__(self):
        self._authors: List['Author'] = []

    def add(self, author: 'Author') -> int:
        author.id = len(self._authors)
        self._authors.append(author)
        return author.id

    def get_by_id(self, author_id: int) -> Optional['Author']:
        if 0 <= author_id < len(self._authors):
            return self._authors[author_id]
        return None

    def delete(self, author_id: int) -> bool:
        if 0 <= author_id < len(self._authors):
            self._authors.pop(author_id)
            self._update_ids()
            return True
        return False

    def get_all(self) -> List['Author']:
        return self._authors.copy()

    def find_by_name(self, name: str) -> List['Author']:
        return [a for a in self._authors if a.name == name]

    def find_by_surname(self, surname: str) -> List['Author']:
        return [a for a in self._authors if a.surname == surname]

    def find_by_year_range(self, from_year: int, to_year: int) ->
List['Author']:
        return [a for a in self._authors if

```

```

        (isinstance(a.birth_year, int) and from_year <=
a.birth_year <= to_year) or
        (isinstance(a.death_year, int) and from_year <=
a.death_year <= to_year)]

    def clear(self):
        self._authors.clear()

    def _update_ids(self):
        for i, author in enumerate(self._authors):
            author.id = i

class Picture:
    def __init__(self,
        name: str = 'Неизвестно',
        authors_ids: List[int] = None,
        main_author_id: Any = 'Неизвестно',
        year: Any = 'Неизвестно',
        price: Any = 'Неизвестно',
        country: str = 'Неизвестно',
        size: str = 'Неизвестно',
        is_available: bool = False,
        **kwargs):
        self.id: int = -1
        self.name = name
        self.authors_ids = authors_ids if authors_ids is not None else []
        self.main_author_id = main_author_id
        self.year = year
        self.price = price
        self.country = country
        self.size = size
        self.is_available = is_available
        self.additional_data = kwargs

class Author:
    def __init__(self,
        surname: str = 'Неизвестно',
        name: str = 'Неизвестно',
        last_name: str = 'Неизвестно',
        birth_year: Any = 'Неизвестно',
        death_year: Any = 'Неизвестно',
        pictures_ids: List[int] = None,
        **kwargs):
        self.id: int = -1
        self.surname = surname
        self.name = name

```

```

        self.last_name = last_name
        self.birth_year = birth_year
        self.death_year = death_year
        self.pictures_ids = pictures_ids if pictures_ids is not None else
[]
        self.additional_data = kwargs

class PictureService:
    def __init__(self, picture_repo: PictureRepository, author_repo:
AuthorRepository):
        self.picture_repo = picture_repo
        self.author_repo = author_repo

    def create_picture(self, name: str = 'Неизвестно', year: Any =
'Неизвестно',
                        price: Any = 'Неизвестно', country: str =
'Неизвестно',
                        size: str = 'Неизвестно', is_available: bool =
False,
                        **kwargs) -> int:
        picture = Picture(name=name, year=year, price=price,
country=country,
                        size=size, is_available=is_available, **kwargs)
        return self.picture_repo.add(picture)

    def delete_picture(self, picture_id: int) -> bool:
        return self.picture_repo.delete(picture_id)

    def add_author_to_picture(self, picture_id: int, author_id: int) ->
bool:
        picture = self.picture_repo.get_by_id(picture_id)
        author = self.author_repo.get_by_id(author_id)

        if picture is None or author is None:
            return False

        if author_id not in picture.authors_ids:
            picture.authors_ids.append(author_id)

        if picture_id not in author.pictures_ids:
            author.pictures_ids.append(picture_id)

        if len(picture.authors_ids) == 1:
            picture.main_author_id = author_id

        return True

```

```

def set_main_author(self, picture_id: int, author_id: int) -> bool:
    picture = self.picture_repo.get_by_id(picture_id)
    author = self.author_repo.get_by_id(author_id)

    if picture is None or author is None:
        return False

    if author_id not in picture.authors_ids:
        picture.authors_ids.append(author_id)

    picture.main_author_id = author_id

    if picture_id not in author.pictures_ids:
        author.pictures_ids.append(picture_id)

    return True

def get_pictures_by_author(self, surname: str, name: str) ->
List[Picture]:
    authors = self.author_repo.find_by_surname(surname)
    if authors:
        author_id = authors[0].id
        return self.picture_repo.find_by_author(author_id)
    return []

class AuthorService:
    def __init__(self, author_repo: AuthorRepository, picture_repo:
PictureRepository):
        self.author_repo = author_repo
        self.picture_repo = picture_repo

    def create_author(self, surname: str = 'Неизвестно', name: str =
'Неизвестно',
                    last_name: str = 'Неизвестно', birth_year: Any =
'Неизвестно',
                    death_year: Any = 'Неизвестно', **kwargs) -> int:
        author = Author(surname=surname, name=name, last_name=last_name,
                        birth_year=birth_year, death_year=death_year,
                        **kwargs)
        return self.author_repo.add(author)

    def delete_author(self, author_id: int) -> bool:
        return self.author_repo.delete(author_id)

    def add_picture_to_author(self, author_id: int, picture_id: int) ->
bool:
        author = self.author_repo.get_by_id(author_id)

```

```

        picture = self.picture_repo.get_by_id(picture_id)

        if author is None or picture is None:
            return False

        if picture_id not in author.pictures_ids:
            author.pictures_ids.append(picture_id)

        if author_id not in picture.authors_ids:
            picture.authors_ids.append(author_id)

        if len(picture.authors_ids) == 1:
            picture.main_author_id = author_id

        return True

    def get_authors_by_picture(self, picture_name: str) -> List[Author]:
        pictures = self.picture_repo.find_by_name(picture_name)
        if pictures:
            picture = pictures[0]
            result = []
            for author_id in picture.authors_ids:
                author = self.author_repo.get_by_id(author_id)
                if author is not None:
                    result.append(author)
            return result
        return []

class PictureFormatter:
    def __init__(self, author_repo: AuthorRepository):
        self.author_repo = author_repo

    def format_table(self, pictures: List[Picture], title: str =
'Картины') -> str:
        if not pictures:
            return f'\n{title}: не найдено'

        table = PrettyTable()
        table.field_names = ['ID', 'Название', 'Главный автор', 'Год',
'Цена ($)',
                                'Страна', 'Размер', 'Доступна', 'Доп.
данные']
        table.align['Название'] = 'l'
        table.align['Главный автор'] = 'l'
        table.align['Страна'] = 'l'
        table.align['Доп. данные'] = 'l'

```

```

        for picture in pictures:
            author_name = self._get_author_name(picture.main_author_id)
            additional =
self._format_additional_data(picture.additional_data)

            table.add_row([
                picture.id,
                picture.name,
                author_name,
                picture.year,
                f'{picture.price:,}' if isinstance(picture.price, int)
else picture.price,
                picture.country,
                picture.size,
                '✓' if picture.is_available else 'X',
                additional
            ])

        result = f'\n{"=" * 100}\n'
        result += f'{title} (найдено: {len(pictures)})\n'
        result += f'\n{"=" * 100}\n'
        result += str(table)
        return result

    def _get_author_name(self, author_id: Any) -> str:
        if isinstance(author_id, int):
            author = self.author_repo.get_by_id(author_id)
            if author:
                return f"{author.surname} {author.name}"
            return 'Неизвестно'

    def _format_additional_data(self, additional_data: Dict[str, Any]) ->
str:
        if additional_data:
            return ', '.join([f"{k}: {v}" for k, v in
additional_data.items()])
            return '-'

class AuthorFormatter:
    def format_table(self, authors: List[Author], title: str = 'Авторы')
-> str:
        if not authors:
            return f'\n{title}: не найдено'

        table = PrettyTable()
        table.field_names = ['ID', 'Фамилия', 'Имя', 'Отчество', 'Год
рождения',

```



```

        'Год смерти', 'Картин', 'Доп. данные']
    table.align['Фамилия'] = 'l'
    table.align['Имя'] = 'l'
    table.align['Отчество'] = 'l'
    table.align['Доп. данные'] = 'l'

    for author in authors:
        additional =
self._format_additional_data(author.additional_data)

        table.add_row([
            author.id,
            author.surname,
            author.name,
            author.last_name if author.last_name else '-',
            author.birth_year,
            author.death_year,
            len(author.pictures_ids),
            additional
        ])

    result = f'\n{"=" * 100}\n'
    result += f'{title} (найдено: {len(authors)})\n'
    result += f'{"=" * 100}\n'
    result += str(table)
    return result

def _format_additional_data(self, additional_data: Dict[str, Any]) ->
str:
    if additional_data:
        return ', '.join([f'{k}: {v}' for k, v in
additional_data.items()])
    return '-'

class ConsoleUI:
    def __init__(self,
        picture_service: PictureService,
        author_service: AuthorService,
        picture_repo: PictureRepository,
        author_repo: AuthorRepository,
        picture_formatter: PictureFormatter,
        author_formatter: AuthorFormatter):
        self.picture_service = picture_service
        self.author_service = author_service
        self.picture_repo = picture_repo
        self.author_repo = author_repo
        self.picture_formatter = picture_formatter

```

```

self.author_formatter = author_formatter

def run(self):
    while True:
        self._print_menu()
        try:
            choice = int(input('\nВыберите действие: '))
        except ValueError:
            print('Ошибка: введите число!')
            continue

        if choice == 0:
            print('\nAdios!')
            break

        self._handle_choice(choice)

def _print_menu(self):
    print('\n' + '=' * 80)
    print('МЕНЮ'.center(80))
    print('=' * 80)
    print('┌ КАРТИНЫ ────────────────────┐')
    print('│ 1 - Показать все картины      │')
    print('│ 2 - Добавить картину         │')
    print('│ 3 - Удалить картину          │')
    print('│ 4 - Назначить автора картине  │')
    print('│ 5 - Найти по названию        │')
    print('│ 6 - Найти по стране          │')
    print('│ 7 - Найти по году            │')
    print('│ 8 - Найти по цене            │')
    print('│ 9 - Найти по автору          │')
    print('└────────────────────────────────┘')
    print('┌ АВТОРЫ ────────────────────┐')
    print('│ 10 - Показать всех авторов    │')
    print('│ 11 - Добавить автора         │')
    print('│ 12 - Удалить автора          │')
    print('│ 13 - Назначить картину автору │')
    print('│ 14 - Найти по имени          │')
    print('│ 15 - Найти по фамилии        │')
    print('│ 16 - Найти по году жизни     │')
    print('│ 17 - Найти по картине        │')
    print('└────────────────────────────────┘')
    print('│ 0 - Выход                    │')
    print('└────────────────────────────────┘')

def _handle_choice(self, choice: int):
    handlers = {
        1: self._show_all_pictures,

```

```

        2: self._add_picture,
        3: self._delete_picture,
        4: self._assign_author_to_picture,
        5: self._find_pictures_by_name,
        6: self._find_pictures_by_country,
        7: self._find_pictures_by_year,
        8: self._find_pictures_by_price,
        9: self._find_pictures_by_author,
        10: self._show_all_authors,
        11: self._add_author,
        12: self._delete_author,
        13: self._assign_picture_to_author,
        14: self._find_authors_by_name,
        15: self._find_authors_by_surname,
        16: self._find_authors_by_year,
        17: self._find_authors_by_picture,
    }

    handler = handlers.get(choice)
    if handler:
        handler()
    else:
        print('Неверный выбор! Выберите число от 0 до 17.')

    def _show_all_pictures(self):
        pictures = self.picture_repo.get_all()
        print(self.picture_formatter.format_table(pictures, 'Все
картины'))

    def _add_picture(self):
        print('\nДобавление новой картины')
        name = input('Название: ')
        year = input('Год: ')
        year = int(year) if year.isdigit() else year
        price = input('Цена ($): ')
        price = int(price) if price.isdigit() else price
        country = input('Страна: ')
        size = input('Размер: ')
        is_available = input('В наличии? (да/нет): ').lower() in ['да',
'yes', 'y', 'д']

        kwargs = {}
        while True:
            add_more = input('Добавить дополнительное поле? (да/нет):
').lower()
            if add_more in ['да', 'yes', 'y', 'д']:
                key = input('Название поля: ')
                value = input('Значение: ')

```

```

        kwargs[key] = value
    else:
        break

    picture_id = self.picture_service.create_picture(
        name=name, year=year, price=price, country=country,
        size=size, is_available=is_available, **kwargs
    )

    if self.author_repo.get_all():
        add_authors = input('Добавить авторов к картине? (да/нет):
').lower()
        if add_authors in ['да', 'yes', 'y', 'д']:
            authors = self.author_repo.get_all()
            print(self.author_formatter.format_table(authors, 'Список
авторов'))

            while True:
                try:
                    auth_id = input('ID автора (или "стоп" для
завершения): ')

                    if auth_id.lower() in ['стоп', 'stop', 'нет',
'no']:

                        break

                    auth_id = int(auth_id)
                    self.picture_service.add_author_to_picture(picture
e_id, auth_id)

                    print(f'Автор с ID {auth_id} добавлен!')
                except ValueError:
                    print('Ошибка: введите число!')

            print('Картина добавлена!')

    def _delete_picture(self):
        pictures = self.picture_repo.get_all()
        print(self.picture_formatter.format_table(pictures, 'Список
картин'))
        try:
            picture_id = int(input('\nВведите ID картины для удаления:
'))

            if self.picture_service.delete_picture(picture_id):
                print('Картина удалена!')
            else:
                print('Картина с таким ID не найдена!')
        except ValueError:
            print('Ошибка: введите число!')

    def _assign_author_to_picture(self):
        pictures = self.picture_repo.get_all()

```

```

        print(self.picture_formatter.format_table(pictures, 'Список
картин'))
    try:
        pic_id = int(input('ID картины: '))
        authors = self.author_repo.get_all()
        print(self.author_formatter.format_table(authors, 'Список
авторов'))
        auth_id = int(input('ID автора: '))
        self.picture_service.add_author_to_picture(pic_id, auth_id)
        print('Автор добавлен картине!')
    except ValueError:
        print('Ошибка: введите число!')

    def _find_pictures_by_name(self):
        name = input('\nВведите название картины: ')
        pictures = self.picture_repo.find_by_name(name)
        print(self.picture_formatter.format_table(pictures, f'Результаты
поиска: "{name}"'))

    def _find_pictures_by_country(self):
        country = input('\nВведите страну: ')
        pictures = self.picture_repo.find_by_country(country)
        print(self.picture_formatter.format_table(pictures, f'Картины из
страны: {country}'))

    def _find_pictures_by_year(self):
        try:
            year = int(input('\nВведите год: '))
            pictures = self.picture_repo.find_by_year(year)
            print(self.picture_formatter.format_table(pictures, f'Картины
{year} года'))
        except ValueError:
            print('Ошибка: введите число!')

    def _find_pictures_by_price(self):
        try:
            from_price = int(input('\nЦена от ($): '))
            to_price = int(input('Цена до ($): '))
            pictures = self.picture_repo.find_by_price_range(from_price,
to_price)
            print(self.picture_formatter.format_table(
                pictures, f'Картины в диапазоне ${from_price:,} -
${to_price:,}')
            ))
        except ValueError:
            print('Ошибка: введите число!')

    def _find_pictures_by_author(self):

```

```

        print('\nПоиск картин по автору')
        surname = input('Фамилия автора: ')
        name = input('Имя автора: ')
        pictures = self.picture_service.get_pictures_by_author(surname,
name)
        print(self.picture_formatter.format_table(
            pictures, f'Картины автора: {surname} {name}'
        ))

    def _show_all_authors(self):
        authors = self.author_repo.get_all()
        print(self.author_formatter.format_table(authors, 'Все авторы'))

    def _add_author(self):
        print('\nДобавление нового автора')
        surname = input('Фамилия: ')
        name = input('Имя: ')
        last_name = input('Отчество: ')
        birth_year = input('Год рождения: ')
        birth_year = int(birth_year) if birth_year.isdigit() else
birth_year
        death_year = input('Год смерти: ')
        death_year = int(death_year) if death_year.isdigit() else
death_year

        kwargs = {}
        while True:
            add_more = input('Добавить дополнительное поле? (да/нет):
').lower()
            if add_more in ['да', 'yes', 'y', 'д']:
                key = input('Название поля: ')
                value = input('Значение: ')
                kwargs[key] = value
            else:
                break

        author_id = self.author_service.create_author(
            surname=surname, name=name, last_name=last_name,
            birth_year=birth_year, death_year=death_year, **kwargs
        )

        if self.picture_repo.get_all():
            add_pictures = input('Добавить картины автору? (да/нет):
').lower()
            if add_pictures in ['да', 'yes', 'y', 'д']:
                pictures = self.picture_repo.get_all()
                print(self.picture_formatter.format_table(pictures,
'Список картин'))

```

```

        while True:
            try:
                pic_id = input('ID картины (или "стоп" для
завершения): ')
                if pic_id.lower() in ['стоп', 'stop', 'нет',
'no']:
                    break
                pic_id = int(pic_id)
                self.author_service.add_picture_to_author(author_
id, pic_id)
                print(f'Картина с ID {pic_id} добавлена!')
            except ValueError:
                print('Ошибка: введите число!')

        print('Автор добавлен!')

    def _delete_author(self):
        authors = self.author_repo.get_all()
        print(self.author_formatter.format_table(authors, 'Список
авторов'))
        try:
            author_id = int(input('\nВведите ID автора для удаления: '))
            if self.author_service.delete_author(author_id):
                print('Автор удален!')
            else:
                print('Автор с таким ID не найден!')
        except ValueError:
            print('Ошибка: введите число!')

    def _assign_picture_to_author(self):
        authors = self.author_repo.get_all()
        print(self.author_formatter.format_table(authors, 'Список
авторов'))
        try:
            auth_id = int(input('ID автора: '))
            pictures = self.picture_repo.get_all()
            print(self.picture_formatter.format_table(pictures, 'Список
картин'))
            pic_id = int(input('ID картины: '))
            self.author_service.add_picture_to_author(auth_id, pic_id)
            print('Картина добавлена автору!')
        except ValueError:
            print('Ошибка: введите число!')

    def _find_authors_by_name(self):
        name = input('\nВведите имя автора: ')
        authors = self.author_repo.find_by_name(name)

```

```

        print(self.author_formatter.format_table(authors, f'Результаты
поиска: "{name}"'))

    def _find_authors_by_surname(self):
        surname = input('\nВведите фамилию автора: ')
        authors = self.author_repo.find_by_surname(surname)
        print(self.author_formatter.format_table(authors, f'Результаты
поиска: "{surname}"'))

    def _find_authors_by_year(self):
        try:
            from_year = int(input('\nГод от: '))
            to_year = int(input('Год до: '))
            authors = self.author_repo.find_by_year_range(from_year,
to_year)
            print(self.author_formatter.format_table(authors, f'Авторы
{from_year}-{to_year} гг.'))
        except ValueError:
            print('Ошибка: введите число!')

    def _find_authors_by_picture(self):
        name = input('\nВведите название картины: ')
        authors = self.author_service.get_authors_by_picture(name)
        print(self.author_formatter.format_table(authors, f'Авторы
картины: "{name}"'))

def create_app() -> ConsoleUI:
    picture_repo = PictureRepository()
    author_repo = AuthorRepository()
    picture_service = PictureService(picture_repo, author_repo)
    author_service = AuthorService(author_repo, picture_repo)
    picture_formatter = PictureFormatter(author_repo)
    author_formatter = AuthorFormatter()

    return ConsoleUI(
        picture_service=picture_service,
        author_service=author_service,
        picture_repo=picture_repo,
        author_repo=author_repo,
        picture_formatter=picture_formatter,
        author_formatter=author_formatter
    )

def seed_data(picture_service: PictureService, author_service:
AuthorService):
    picture_service.create_picture(

```



```

        name='Мона Лиза', year=1503, price=570000,
        country='Италия', size='77x53 см', is_available=True,
техника='Масло'
    )
    picture_service.create_picture(
        name='Звёздная ночь', year=1889, price=80000,
        country='Нидерланды', size='73x92 см', техника='Масло'
    )
    picture_service.create_picture(
        name='Девятый вал', year=1850, price=123000,
        country='Россия', size='221x332 см', is_available=True,
техника='Масло'
    )
    picture_service.create_picture(
        name='Сотворение Адама', year=1512, price=10000,
        country='Италия', size='280x570 см', is_available=True,
техника='Фреска'
    )
    picture_service.create_picture(
        name='Чёрный квадрат', year=1915, price=19000,
        country='Россия', size='79x79 см', стиль='Супрематизм'
    )

    author_service.create_author(
        surname='да Винчи', name='Леонардо', last_name='',
        birth_year=1452, death_year=1519, страна='Италия'
    )
    author_service.create_author(
        surname='ван Гог', name='Винсент', last_name='',
        birth_year=1853, death_year=1890, страна='Нидерланды'
    )
    author_service.create_author(
        surname='Айвазовский', name='Иван', last_name='Константинович',
        birth_year=1817, death_year=1900, страна='Россия'
    )
    author_service.create_author(
        surname='Буонарроти', name='Микеланджело', last_name='',
        birth_year=1475, death_year=1564, страна='Италия'
    )
    author_service.create_author(
        surname='Малевич', name='Казимир', last_name='Северинович',
        birth_year=1879, death_year=1935, страна='Россия'
    )

    for i in range(5):
        picture_service.add_author_to_picture(i, i)

```

```
if __name__ == '__main__':
    app = create_app()
    seed_data(app.picture_service, app.author_service)
    app.run()
```

test_art_gallery.py

```
import unittest
from prettytable import PrettyTable
from typing import List
import sys

from art_gallery import (
    Picture, Author,
    PictureRepository, AuthorRepository,
    PictureService, AuthorService,
    PictureFormatter, AuthorFormatter
)

class BeautifulTestResult(unittest.TextTestResult):

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.test_results = []
        self.current_test_class = None

    def startTest(self, test):
        super().startTest(test)
        self.current_test_start = test

    def addSuccess(self, test):
        super().addSuccess(test)
        self.test_results.append({
            'class': test.__class__.__name__,
            'test': test._testMethodName,
            'status': '✓ PASS',
            'message': ''
        })

    def addError(self, test, err):
        super().addError(test, err)
        self.test_results.append({
            'class': test.__class__.__name__,
            'test': test._testMethodName,
            'status': 'X ERROR',
            'message': str(err[1])
        })
```

```

def addFailure(self, test, err):
    super().addFailure(test, err)
    self.test_results.append({
        'class': test.__class__.__name__,
        'test': test._testMethodName,
        'status': 'X FAIL',
        'message': str(err[1])
    })

def addSkip(self, test, reason):
    super().addSkip(test, reason)
    self.test_results.append({
        'class': test.__class__.__name__,
        'test': test._testMethodName,
        'status': 'o SKIP',
        'message': reason
    })

def print_beautiful_results(self):
    print('\n' + '=' * 120)
    print('ИТОГОВЫЕ РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ'.center(120))
    print('=' * 120)

    test_classes = {}
    for result in self.test_results:
        class_name = result['class']
        if class_name not in test_classes:
            test_classes[class_name] = []
        test_classes[class_name].append(result)

    for class_name, tests in test_classes.items():
        print(f'\n{class_name}')
        print('-' * 120)

        table = PrettyTable()
        table.field_names = ['№', 'Название теста', 'Статус',
'Сообщение об ошибке']
        table.align['№'] = 'c'
        table.align['Название теста'] = 'l'
        table.align['Статус'] = 'c'
        table.align['Сообщение об ошибке'] = 'l'
        table.max_width['Название теста'] = 60
        table.max_width['Сообщение об ошибке'] = 40

        for idx, test in enumerate(tests, 1):
            test_name = test['test'].replace('test_',
'').replace('_', ' ').capitalize()

```

```

        status = test['status']
        message = test['message'][:100] if test['message'] else
        '_'

        table.add_row([idx, test_name, status, message])

    print(table)

    print('\n' + '=' * 120)
    self._print_summary()
    print('=' * 120 + '\n')

    def _print_summary(self):
        total = len(self.test_results)
        passed = len([r for r in self.test_results if '✓ PASS' in
r['status']])
        failed = len([r for r in self.test_results if 'X FAIL' in
r['status']])
        errors = len([r for r in self.test_results if 'X ERROR' in
r['status']])
        skipped = len([r for r in self.test_results if 'o SKIP' in
r['status']])

        summary_table = PrettyTable()
        summary_table.field_names = ['Всего', 'Пройдено', 'Провалено',
'Ошибок', 'Пропущено', 'Успешность']

        success_rate = (passed / total * 100) if total > 0 else 0

        summary_table.add_row([
            total,
            f'{passed} ({passed/total*100:.1f}%)' if total > 0 else '0',
            f'{failed} ({failed/total*100:.1f}%)' if total > 0 else '0',
            f'{errors} ({errors/total*100:.1f}%)' if total > 0 else '0',
            f'{skipped} ({skipped/total*100:.1f}%)' if total > 0 else
'0',
            f'{success_rate:.1f}%'
        ])

    print('\nСТАТИСТИКА')
    print(summary_table)

    if success_rate == 100:
        print('\nРЕЗУЛЬТАТ: ✓ Все тесты успешно
пройжены'.center(120))
    elif success_rate >= 80:

```

```

        print('\nРЕЗУЛЬТАТ: ~ Большинство тестов пройдено, есть
замечания'.center(120))
    else:
        print('\nРЕЗУЛЬТАТ: X Требуется исправление критических
ошибок'.center(120))

class BeautifulTestRunner(unittest.TextTestRunner):
    resultclass = BeautifulTestResult

    def run(self, test):
        print('\n' + '=' * 120)
        print('ЗАПУСК НАБОРА ТЕСТОВ'.center(120))
        print('=' * 120)
        print('\nПосле каждого теста нажимайте ENTER для продолжения\n')

        result = super().run(test)

        if hasattr(result, 'print_beautiful_results'):
            result.print_beautiful_results()

        return result

class TestPictureRepository(unittest.TestCase):

    @classmethod
    def setUpClass(cls):
        print(f'\n{"=" * 80}')
        print(f'КЛАСС ТЕСТОВ: {cls.__name__}'.center(80))
        print(f'{"=" * 80}')
```

```

    def setUp(self):
        self.repo = PictureRepository()
        print(f'\n{"-" * 80}')
```

```

        print(f'► Тест: {self._testMethodName}')
```

```

        print(f'{"-" * 80}')
```

```

    def tearDown(self):
        print(f'\n{"-" * 80}')
```

```

        print(f'✓ Тест завершён: {self._testMethodName}')
```

```

        print(f'{"-" * 80}')
```

```

        input('\n[Нажмите ENTER для следующего теста]')
```

```

    def test_add_picture_assigns_correct_id(self):
        print('\nОписание: Проверяем, что репозиторий последовательно
присваивает ID')
```

```

        picture1 = Picture(name='Мона Лиза', year=1503)
```

```

picture2 = Picture(name='Звёздная ночь', year=1889)

id1 = self.repo.add(picture1)
id2 = self.repo.add(picture2)

print('\nРезультат операций:')
self._print_test_data([picture1, picture2])

self.assertEqual(id1, 0, "Первая картина должна иметь ID = 0")
self.assertEqual(id2, 1, "Вторая картина должна иметь ID = 1")
self.assertEqual(picture1.id, 0, "ID должен быть присвоен
объекту")
self.assertEqual(picture2.id, 1, "ID должен быть присвоен
объекту")

print('\nПроверки:')
print('  ✓ ID присваиваются последовательно (0, 1, 2...)')
print('  ✓ ID сохраняется в объекте картины')

def test_delete_picture_updates_ids(self):
    print('\nОписание: При удалении картины ID остальных должны
пересчитаться')

    picture1 = Picture(name='Картина 1')
    picture2 = Picture(name='Картина 2')
    picture3 = Picture(name='Картина 3')

    self.repo.add(picture1)
    self.repo.add(picture2)
    self.repo.add(picture3)

    print('\nСостояние ДО удаления:')
    self._print_test_data(self.repo.get_all())

    success = self.repo.delete(1)

    print('\nСостояние ПОСЛЕ удаления картины с ID=1:')
    self._print_test_data(self.repo.get_all())

    self.assertTrue(success, "Удаление должно быть успешным")
    self.assertEqual(len(self.repo.get_all()), 2, "Должно остаться 2
картины")

    remaining_pictures = self.repo.get_all()
    self.assertEqual(remaining_pictures[0].id, 0, "Первая картина:
ID=0")
    self.assertEqual(remaining_pictures[1].id, 1, "Третья картина
стала второй: ID=1")

```

```

        self.assertEqual(remaining_pictures[1].name, 'Картина 3', "Это
должна быть 'Картина 3'")

    print('\nПроверки:')
    print('  ✓ Картина с ID=1 удалена')
    print('  ✓ ID оставшихся картин пересчитаны')
    print('  ✓ Порядок картин сохранён')

def test_find_by_price_range(self):
    print('\nОписание: Поиск картин в заданном диапазоне цен')

    self.repo.add(Picture(name='Дешёвая', price=1000))
    self.repo.add(Picture(name='Средняя', price=5000))
    self.repo.add(Picture(name='Дорогая', price=10000))
    self.repo.add(Picture(name='Неизвестная цена',
price='Неизвестно'))

    print('\nВсе картины в базе:')
    self._print_test_data(self.repo.get_all())

    print('\nПараметры поиска: от 2000$ до 8000$')
    results = self.repo.find_by_price_range(2000, 8000)

    print('\nРезультаты поиска:')
    self._print_test_data(results)

    self.assertEqual(len(results), 1, "Должна найтись 1 картина")
    self.assertEqual(results[0].name, 'Средняя', "Должна найтись
'Средняя'")

    print('\nПроверки:')
    print('  ✓ Найдена 1 картина в диапазоне')
    print('  ✓ Картины с некорректной ценой отфильтрованы')

def _print_test_data(self, pictures: List[Picture]):
    if not pictures:
        print('  (пусто)')
        return

    table = PrettyTable()
    table.field_names = ['ID', 'Название', 'Год', 'Цена']
    table.align['Название'] = 'l'
    table.align['ID'] = 'c'

    for pic in pictures:
        table.add_row([
            pic.id,
            pic.name,

```

```

        pic.year if hasattr(pic, 'year') else '-',
        f'{pic.price}$' if isinstance(pic.price, int) else
pic.price
    ])

    print(table)

```

```

class TestPictureService(unittest.TestCase):

```

```

    @classmethod
    def setUpClass(cls):
        print(f'\n{"=" * 80}')
        print(f'КЛАСС ТЕСТОВ: {cls.__name__}'.center(80))
        print(f'{"=" * 80}')

    def setUp(self):
        self.picture_repo = PictureRepository()
        self.author_repo = AuthorRepository()
        self.service = PictureService(self.picture_repo,
self.author_repo)
        print(f'\n{"-" * 80}')
        print(f'▶ ТЕСТ: {self._testMethodName}')
        print(f'{"-" * 80}')

    def tearDown(self):
        print(f'\n{"-" * 80}')
        print(f'✓ Тест завершён: {self._testMethodName}')
        print(f'{"-" * 80}')
        input('\n[Нажмите ENTER для следующего теста]')

    def test_add_author_to_picture_creates_bidirectional_link(self):
        print('\nОписание: Связывание должно обновить списки как у
картины, так и у автора')

        picture_id = self.service.create_picture(name='Мона Лиза')
        author = Author(surname='да Винчи', name='Леонардо')
        author_id = self.author_repo.add(author)

        print('\nСостояние ДО связывания:')
        self._print_relationship_status(picture_id, author_id)

        success = self.service.add_author_to_picture(picture_id,
author_id)

        print('\nСостояние ПОСЛЕ связывания:')
        self._print_relationship_status(picture_id, author_id)

```



```

        self.assertTrue(success, "Операция должна быть успешной")

        picture = self.picture_repo.get_by_id(picture_id)
        self.assertIn(author_id, picture.authors_ids, "ID автора должен
        быть в списке авторов картины")
        self.assertEqual(picture.main_author_id, author_id, "Автор должен
        стать главным (первый)")

        author = self.author_repo.get_by_id(author_id)
        self.assertIn(picture_id, author.pictures_ids, "ID картины должен
        быть в списке картин автора")

        print('\nПроверки:')
        print('    ✓ Двусторонняя связь установлена')
        print('    ✓ Автор назначен главным')

    def test_add_author_to_picture_with_invalid_ids(self):
        print('\nОписание: При попытке связать несуществующие объекты
        должна быть ошибка')

        picture_id = self.service.create_picture(name='Картина')
        invalid_author_id = 999

        print(f'\nПопытка связать:')
        print(f'    Картина: ID={picture_id} (существует)')
        print(f'    Автор: ID={invalid_author_id} (НЕ существует)')

        success = self.service.add_author_to_picture(picture_id,
        invalid_author_id)

        self.assertFalse(success, "Операция должна завершиться неудачей")

        picture = self.picture_repo.get_by_id(picture_id)
        self.assertEqual(len(picture.authors_ids), 0, "У картины не
        должно быть авторов")

        print('\nРезультат: X Операция корректно отклонена')
        print('\nПроверки:')
        print('    ✓ Метод вернул False')
        print('    ✓ Картина не изменилась')

    def test_second_author_does_not_become_main(self):
        print('\nОписание: При добавлении второго автора главный не
        должен меняться')

        picture_id = self.service.create_picture(name='Картина')

        author1 = Author(surname='Первый', name='Автор')

```

```

author2 = Author(surname='Второй', name='Автор')

author1_id = self.author_repo.add(author1)
author2_id = self.author_repo.add(author2)

self.service.add_author_to_picture(picture_id, author1_id)

print('\nПосле добавления первого автора:')
self._print_relationship_status(picture_id, author1_id)

self.service.add_author_to_picture(picture_id, author2_id)

print('\nПосле добавления второго автора:')
self._print_relationship_status(picture_id, author2_id)

picture = self.picture_repo.get_by_id(picture_id)
self.assertEqual(len(picture.authors_ids), 2, "Должно быть 2
автора")
self.assertEqual(picture.main_author_id, author1_id, "Главный
автор – первый добавленный")
self.assertNotEqual(picture.main_author_id, author2_id, "Второй
автор не должен быть главным")

print('\nПроверки:')
print(f'  ✓ У картины 2 автора')
print(f'  ✓ Главный автор: ID={author1_id} (первый)')
print(f'  ✓ Второй автор не стал главным')

def _print_relationship_status(self, picture_id: int, author_id:
int):
    picture = self.picture_repo.get_by_id(picture_id)
    author = self.author_repo.get_by_id(author_id)

    table = PrettyTable()
    table.field_names = ['Тип', 'Объект', 'Связанные ID', 'Главный
автор']
    table.align['Объект'] = 'l'
    table.align['Связанные ID'] = 'l'
    table.align['Тип'] = 'c'

    if picture:
        table.add_row([
            'Картина',
            f'{picture.name} (ID={picture.id})',
            f'Авторы: {picture.authors_ids}' if picture.authors_ids
else 'Нет связей',
            f'ID={picture.main_author_id}' if picture.main_author_id
!= 'Неизвестно' else 'Не назначен'

```

```

    ])

    if author:
        table.add_row([
            'Автор',
            f'{author.surname} {author.name} (ID={author.id})',
            f'Картины: {author.pictures_ids}' if author.pictures_ids
else 'Нет связей',
            '_'
        ])

    print(table)

```

```

class TestPictureFormatter(unittest.TestCase):

```

```

    @classmethod
    def setUpClass(cls):
        print(f'\n{"=" * 80}')
        print(f'КЛАСС ТЕСТОВ: {cls.__name__}'.center(80))
        print(f'{"=" * 80}')
```

```

    def setUp(self):
        self.author_repo = AuthorRepository()
        self.formatter = PictureFormatter(self.author_repo)
        print(f'\n{"-" * 80}')
```

```

        print(f'► Тест: {self._testMethodName}')
        print(f'{"-" * 80}')
```

```

    def tearDown(self):
        print(f'\n{"-" * 80}')
```

```

        print(f'✓ Тест завершён: {self._testMethodName}')
        print(f'{"-" * 80}')
```

```

        input('\n[Нажмите ENTER для следующего теста]')
```

```

    def test_format_empty_list(self):
        """Тест: форматирование пустого списка"""
        print('\nОписание: Пустой список должен быть обработан
корректно')

        pictures = []

        result = self.formatter.format_table(pictures, 'Тестовые
картины')

        print('\nРезультат форматирования:')
        print(result)

```

```

        self.assertIn('не найдено', result, "Должно быть сообщение 'не
        найдено'")
        self.assertIn('Тестовые картины', result, "Должен быть
        заголовок")

        print('\nПроверки:')
        print('  ✓ Есть сообщение "не найдено"')
        print('  ✓ Заголовок отображается')

    def test_format_picture_with_author(self):
        """Тест: форматирование картины с автором"""
        print('\nОписание: Картина с автором должна отображаться
        полностью')

        author = Author(surname='Пикассо', name='Пабло')
        author_id = self.author_repo.add(author)

        picture = Picture(
            name='Герника',
            year=1937,
            price=200000,
            country='Испания',
            size='349x776 см',
            is_available=True,
            main_author_id=author_id
        )
        picture.id = 0

        result = self.formatter.format_table([picture], 'Картины
        Пикассо')

        print('\nОтформатированная таблица:')
        print(result)

        self.assertIn('Герника', result, "Должно быть название картины")
        self.assertIn('Пикассо Пабло', result, "Должно быть имя автора")
        self.assertIn('1937', result, "Должен быть год")
        self.assertIn('Испания', result, "Должна быть страна")
        self.assertIn('✓', result, "Должна быть галочка доступности")

        print('\nПроверки:')
        print('  ✓ Название картины отображено')
        print('  ✓ Имя автора получено по ID')
        print('  ✓ Все поля заполнены')

    def test_format_picture_with_unknown_author(self):
        print('\nОписание: Картина без автора должна отображать
        "Неизвестно"')

```

```

        picture = Picture(name='Неизвестная картина',
main_author_id='Неизвестно')
        picture.id = 0

        result = self.formatter.format_table([picture])

        print('\nОтформатированная таблица:')
        print(result)

        self.assertIn('Неизвестно', result, "Автор должен быть
'Неизвестно'")
        self.assertIn('Неизвестная картина', result, "Должно быть
название")

        print('\nПроверки:')
        print('  ✓ Название картины отображено')
        print('  ✓ Автор указан как "Неизвестно"')

if __name__ == '__main__':
    loader = unittest.TestLoader()
    suite = unittest.TestSuite()

    suite.addTests(loader.loadTestsFromTestCase(TestPictureRepository))
    suite.addTests(loader.loadTestsFromTestCase(TestPictureService))
    suite.addTests(loader.loadTestsFromTestCase(TestPictureFormatter))

    runner = BeautifulTestRunner(verbosity=2)
    result = runner.run(suite)

    sys.exit(0 if result.wasSuccessful() else 1)

```

Анализ результатов

1. Тест 1. Присваивание ID.

```
=====
                                ЗАПУСК НАБОРА ТЕСТОВ
=====

После каждого теста нажимайте ENTER для продолжения

=====
                                КЛАСС ТЕСТОВ: TestPictureRepository
=====
test_add_picture_assigns_correct_id (__main__.TestPictureRepository.test_add_picture_assigns_correct_id) ...
> Тест: test_add_picture_assigns_correct_id
=====

Описание: Проверяем, что репозиторий последовательно присваивает ID

Результат операций:
+-----+-----+-----+-----+
| ID | Название | Год | Цена |
+-----+-----+-----+-----+
| 0 | Мона Лиза | 1503 | Неизвестно |
| 1 | Звёздная ночь | 1889 | Неизвестно |
+-----+-----+-----+-----+

Проверки:
✓ ID присваиваются последовательно (0, 1, 2...)
✓ ID сохраняется в объекте картины

✓ Тест завершён: test_add_picture_assigns_correct_id
```

2. Тест 2. Пересчитывание ID.

```
test_delete_picture_updates_ids (__main__.TestPictureRepository.test_delete_picture_updates_ids) ...
> Тест: test_delete_picture_updates_ids
=====

Описание: При удалении картины ID остальных должны пересчитаться

Состояние ДО удаления:
+-----+-----+-----+-----+
| ID | Название | Год | Цена |
+-----+-----+-----+-----+
| 0 | Картина 1 | Неизвестно | Неизвестно |
| 1 | Картина 2 | Неизвестно | Неизвестно |
| 2 | Картина 3 | Неизвестно | Неизвестно |
+-----+-----+-----+-----+

Состояние ПОСЛЕ удаления картины с ID=1:
+-----+-----+-----+-----+
| ID | Название | Год | Цена |
+-----+-----+-----+-----+
| 0 | Картина 1 | Неизвестно | Неизвестно |
| 1 | Картина 3 | Неизвестно | Неизвестно |
+-----+-----+-----+-----+

Проверки:
✓ Картина с ID=1 удалена
✓ ID оставшихся картин пересчитаны
✓ Порядок картин сохранён

✓ Тест завершён: test_delete_picture_updates_ids
```

3. Тест 3. Поиск картин с определённой ценой.

```
test_find_by_price_range (__main__.TestPictureRepository.test_find_by_price_range) ...
```

```
► Тест: test_find_by_price_range
```

Описание: Поиск картин в заданном диапазоне цен

Все картины в базе:

ID	Название	Год	Цена
0	Дешёвая	Неизвестно	1000\$
1	Средняя	Неизвестно	5000\$
2	Дорогая	Неизвестно	10000\$
3	Неизвестная цена	Неизвестно	Неизвестно

Параметры поиска: от 2000\$ до 8000\$

Результаты поиска:

ID	Название	Год	Цена
1	Средняя	Неизвестно	5000\$

Проверки:

- ✓ Найдена 1 картина в диапазоне
- ✓ Картины с некорректной ценой отфильтрованы

✓ Тест завершён: test_find_by_price_range

4. Тест 4. Связывание картины и автора.

```
КЛАСС ТЕСТОВ: TestPictureService
```

```
test_add_author_to_picture_creates_bidirectional_link (__main__.TestPictureService.test_add_author_to_picture_creates_bidirectional_link) ...
```

```
► Тест: test_add_author_to_picture_creates_bidirectional_link
```

Описание: Связывание должно обновить списки как у картины, так и у автора

Состояние ДО связывания:

Тип	Объект	Связанные ID	Главный автор
Картина	Мона Лиза (ID=0)	Нет связей	Не назначен
Автор	да Винчи Леонардо (ID=0)	Нет связей	—

Состояние ПОСЛЕ связывания:

Тип	Объект	Связанные ID	Главный автор
Картина	Мона Лиза (ID=0)	Авторы: [0]	ID=0
Автор	да Винчи Леонардо (ID=0)	Картины: [0]	—

Проверки:

- ✓ Двусторонняя связь установлена
- ✓ Автор назначен главным

✓ Тест завершён: test_add_author_to_picture_creates_bidirectional_link

5. Тест 5. Связывание несуществующих объектов.

```
test_add_author_to_picture_with_invalid_ids (__main__.TestPictureService.test_add_author_to_picture_with_invalid_ids) ...
└─ Тест: test_add_author_to_picture_with_invalid_ids
-----

Описание: При попытке связать несуществующие объекты должна быть ошибка

Попытка связать:
  Картина: ID=0 (существует)
  Автор: ID=999 (НЕ существует)

Результат: X Операция корректно отклонена

Проверки:
  ✓ Метод вернул False
  ✓ Картина не изменилась

-----
✓ Тест завершён: test_add_author_to_picture_with_invalid_ids
-----
```

6. Тест 6. Добавление второго автора.

```
test_second_author_does_not_become_main (__main__.TestPictureService.test_second_author_does_not_become_main) ...
└─ Тест: test_second_author_does_not_become_main
-----

Описание: При добавлении второго автора главный не должен меняться

После добавления первого автора:
+-----+-----+-----+-----+
| Тип | Объект | Связанные ID | Главный автор |
+-----+-----+-----+-----+
| Картина | Картина (ID=0) | Авторы: [0] | ID=0 |
| Автор | Первый Автор (ID=0) | Картины: [0] | — |
+-----+-----+-----+-----+

После добавления второго автора:
+-----+-----+-----+-----+
| Тип | Объект | Связанные ID | Главный автор |
+-----+-----+-----+-----+
| Картина | Картина (ID=0) | Авторы: [0, 1] | ID=0 |
| Автор | Второй Автор (ID=1) | Картины: [0] | — |
+-----+-----+-----+-----+

Проверки:
  ✓ У картины 2 автора
  ✓ Главный автор: ID=0 (первый)
  ✓ Второй автор не стал главным

-----
✓ Тест завершён: test_second_author_does_not_become_main
-----
```

7. Тест 7. Обработка пустого списка.


```

=====
                        КЛАСС ТЕСТОВ: TestPictureFormatter
=====
test_format_empty_list (__main__.TestPictureFormatter.test_format_empty_list)
Тест: форматирование пустого списка ...
-----
▶ Тест: test_format_empty_list
-----

Описание: Пустой список должен быть обработан корректно

Результат форматирования:

Тестовые картины: не найдено

Проверки:
  ✓ Есть сообщение "не найдено"
  ✓ Заголовок отображается

-----
✓ Тест завершён: test_format_empty_list
-----

```

8. Тест 8. Отображение информации о картине.

```

test_format_picture_with_author (__main__.TestPictureFormatter.test_format_picture_with_author)
Тест: форматирование картины с автором ...
-----
▶ Тест: test_format_picture_with_author
-----

Описание: Картина с автором должна отображаться полностью

Отформатированная таблица:

=====
Картины Пикассо (найдено: 1)
=====
+---+-----+-----+-----+-----+-----+-----+-----+-----+
| ID | Название | Главный автор | Год | Цена ($) | Страна | Размер | Доступна | Доп. данные |
+---+-----+-----+-----+-----+-----+-----+-----+-----+
| 0  | Герника  | Пикассо Пабло | 1937 | 200,000  | Испания | 349x776 см | ✓        | -          |
+---+-----+-----+-----+-----+-----+-----+-----+-----+

Проверки:
  ✓ Название картины отображено
  ✓ Имя автора получено по ID
  ✓ Все поля заполнены

-----
✓ Тест завершён: test_format_picture_with_author
-----

```

9. Тест 9. Отображение информации о картине без автора.

```
test_format_picture_with_unknown_author ( _main_.TestPictureFormatter.test_format_picture_with_unknown_author) ...
-----
► Тест: test_format_picture_with_unknown_author
-----

Описание: Картина без автора должна отображать "Неизвестно"

Отформатированная таблица:
=====
Картины (найдено: 1)
=====
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| ID | Название | Главный автор | Год | Цена ($) | Страна | Размер | Доступна | Доп. данные |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 0 | Неизвестная картина | Неизвестно | Неизвестно | Неизвестно | Неизвестно | Неизвестно | X | - |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+

Проверки:
✓ Название картины отображено
✓ Автор указан как "Неизвестно"

-----
✓ Тест завершён: test_format_picture_with_unknown_author
-----
```

10. Результаты тестирования.

```
=====
ИТОГОВЫЕ РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ
=====

TestPictureRepository
-----
+-----+-----+-----+-----+
| № | Название теста | Статус | Сообщение об ошибке |
+-----+-----+-----+-----+
| 1 | Add picture assigns correct id | ✓ PASS | - |
| 2 | Delete picture updates ids | ✓ PASS | - |
| 3 | Find by price range | ✓ PASS | - |
+-----+-----+-----+-----+

TestPictureService
-----
+-----+-----+-----+-----+
| № | Название теста | Статус | Сообщение об ошибке |
+-----+-----+-----+-----+
| 1 | Add author to picture creates bidirectional link | ✓ PASS | - |
| 2 | Add author to picture with invalid ids | ✓ PASS | - |
| 3 | Second author does not become main | ✓ PASS | - |
+-----+-----+-----+-----+

TestPictureFormatter
-----
+-----+-----+-----+-----+
| № | Название теста | Статус | Сообщение об ошибке |
+-----+-----+-----+-----+
| 1 | Format empty list | ✓ PASS | - |
| 2 | Format picture with author | ✓ PASS | - |
| 3 | Format picture with unknown author | ✓ PASS | - |
+-----+-----+-----+-----+
```

11. Статистика.

```
=====
СТАТИСТИКА
+-----+-----+-----+-----+-----+-----+
| Всего | Пройдено | Провалено | Ошибок | Пропущено | Успешность |
+-----+-----+-----+-----+-----+-----+
| 9 | 9 (100.0%) | 0 (0.0%) | 0 (0.0%) | 0 (0.0%) | 100.0% |
+-----+-----+-----+-----+-----+-----+

РЕЗУЛЬТАТ: ✓ Все тесты успешно пройдены
=====
```

12. P.S. В файлике cowsay_test.py более интересная реализация

13. P.P.S. С наступающим Новым Годом, Юрий Евгеньевич! :)